

CITYFIX BUMI SHOLAWAT

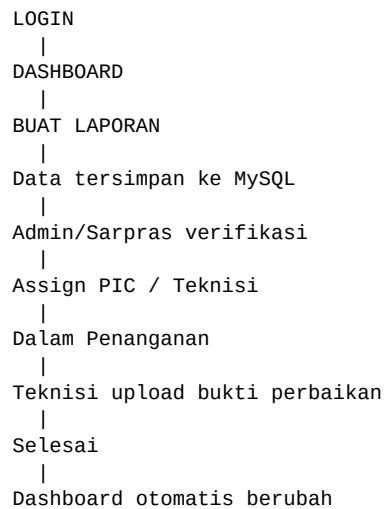
Tahap 2 - Development

Panduan implementasi Laravel + AdminLTE 3 + Bootstrap + MySQL

Dokumen ini melanjutkan prototype Tahap 1 menjadi aplikasi CityFix yang menyimpan data nyata, memiliki autentikasi, role, workflow laporan, upload foto, assignment PIC, status tracking, dan dashboard berbasis database.

1. Target Akhir Tahap 2

Pada akhir Tahap 2, CityFix tidak lagi menggunakan dummy data. Alur utamanya menjadi:



Urutan pengerjaan disarankan:

No	Pekerjaan
1	Setup MySQL
2	Buat database CityFix
3	Konfigurasi Laravel
4	Migration database
5	Model dan relasi
6	Seeder master data
7	Login
8	Role dan middleware
9	CRUD Area
10	CRUD Kategori
11	Report Engine
12	Upload foto
13	Detail laporan
14	Assignment PIC
15	Status tracking
16	Completion proof
17	Dashboard database
18	Testing

2. Setup MySQL

Pada Tahap 1 kita menggunakan SQLite agar prototype cepat dibuat. Di Tahap 2 gunakan MySQL/MariaDB. Anda dapat memakai MySQL Server, Laragon, XAMPP, atau MariaDB. Untuk development Windows, Laragon atau XAMPP biasanya paling mudah.

Pastikan service database aktif, umumnya dengan parameter berikut:

```
Host : 127.0.0.1
Port : 3306
```

3. Membuat Database CityFix

Buka phpMyAdmin, HeidiSQL, atau MySQL client, lalu jalankan:

```
CREATE DATABASE cityfix;
```

Tidak perlu membuat tabel secara manual. Struktur tabel akan dibuat menggunakan Laravel Migration.

4. Konfigurasi .env

Buka file **.env** pada root project CityFix. Ubah konfigurasi database menjadi:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=cityfix
DB_USERNAME=root
DB_PASSWORD=
```

Jika MySQL menggunakan password, isi DB_PASSWORD sesuai konfigurasi Anda. Setelah mengubah **.env**, jalankan:

```
php artisan config:clear
```

5. Test Koneksi MySQL

```
php artisan migrate
```

Jika berhasil, migration bawaan Laravel akan dibuat. Jika muncul **Access denied**, periksa username/password. Jika muncul **Unknown database cityfix**, database belum dibuat atau nama database tidak sesuai.

6. Desain Database CityFix

Tabel inti yang digunakan:

```
users
areas
categories
reports
report_histories
```

Relasi logisnya:

```
USERS
| reporter
v
REPORTS ---- AREA
| \---- CATEGORY
|
+---- assigned technician
|
v
REPORT HISTORIES
```

7. Menambahkan Role pada Users

CityFix menggunakan empat role: admin, verifier, technician, dan reporter.

```
php artisan make:migration add_role_to_users_table --table=users
```

Isi method up():

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('role')
            ->default('reporter')
            ->after('password');
    });
}
```

Method down():

```
public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('role');
    });
}

php artisan migrate
```

8. Membuat Tabel Area

```
php artisan make:model Area -m
```

Migration areas:

```
Schema::create('areas', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('code')->nullable();
    $table->boolean('is_active')->default(true);
    $table->timestamps();
});
```

9. Membuat Tabel Category

```
php artisan make:model Category -m
```

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('icon')->nullable();
    $table->boolean('is_active')->default(true);
    $table->timestamps();
});
```

10. Membuat Tabel Reports

```
php artisan make:model Report -m
```

```
Schema::create('reports', function (Blueprint $table) {
    $table->id();
    $table->string('report_number')->unique();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->foreignId('area_id')->constrained()->restrictOnDelete();
    $table->foreignId('category_id')->constrained()->restrictOnDelete();
    $table->string('location_detail');
    $table->text('description');
    $table->enum('urgency', ['low', 'medium', 'high', 'emergency']);
    $table->enum('status', [
        'reported', 'verified', 'in_progress',
        'waiting_material', 'completed', 'rejected'
    ]->default('reported'));
    $table->string('photo')->nullable();
    $table->foreignId('assigned_to')->nullable()
        ->constrained('users')->nullOnDelete();
    $table->timestamp('reported_at')->nullable();
    $table->timestamp('started_at')->nullable();
    $table->timestamp('completed_at')->nullable();
    $table->timestamps();
});
```

11. Membuat Report History

Report history diperlukan agar aplikasi menyimpan jejak perubahan status, bukan hanya status terakhir.

```
php artisan make:model ReportHistory -m

Schema::create('report_histories', function (Blueprint $table) {
    $table->id();
    $table->foreignId('report_id')->constrained()->cascadeOnDelete();
    $table->foreignId('user_id')->nullable()->constrained()->nullOnDelete();
    $table->string('status');
    $table->text('note')->nullable();
    $table->string('photo')->nullable();
    $table->timestamps();
});
```

Contoh histori yang nantinya tersimpan:

```
26 Sep 08:00  Dilaporkan
26 Sep 08:30  Diverifikasi
26 Sep 09:00  Assign Teknisi
26 Sep 10:15  Dalam Penanganan
26 Sep 12:30  Selesai
```

12. Jalankan Semua Migration

```
php artisan migrate
```

Cek database dan pastikan tabel users, areas, categories, reports, dan report_histories tersedia.

13. Model dan Relasi

13.1 Area.php

```
class Area extends Model
{
    protected $fillable = ['name', 'code', 'is_active'];

    public function reports()
    {
        return $this->hasMany(Report::class);
    }
}
```

13.2 Category.php

```
class Category extends Model
{
    protected $fillable = ['name', 'icon', 'is_active'];

    public function reports()
    {
        return $this->hasMany(Report::class);
    }
}
```

13.3 Report.php

```
class Report extends Model
{
    protected $fillable = [
        'report_number', 'user_id', 'area_id', 'category_id',
        'location_detail', 'description', 'urgency', 'status',
        'photo', 'assigned_to', 'reported_at', 'started_at', 'completed_at'
    ];

    protected function casts(): array
    {
        return [
            'reported_at' => 'datetime',
            'started_at' => 'datetime',
        ];
    }
}
```

```

        'completed_at' => 'datetime',
    ];
}

public function reporter() { return $this->belongsTo(User::class, 'user_id'); }
public function technician() { return $this->belongsTo(User::class, 'assigned_to'); }
public function area() { return $this->belongsTo(Area::class); }
public function category() { return $this->belongsTo(Category::class); }
public function histories() { return $this->hasMany(ReportHistory::class); }
}

```

13.4 ReportHistory.php

```

class ReportHistory extends Model
{
    protected $fillable = ['report_id', 'user_id', 'status', 'note', 'photo'];

    public function report() { return $this->belongsTo(Report::class); }
    public function user() { return $this->belongsTo(User::class); }
}

```

13.5 Update User.php

```

protected $fillable = ['name', 'email', 'password', 'role'];

public function reports()
{
    return $this->hasMany(Report::class);
}

```

14. Seeder Master Data

14.1 Area Seeder

```

php artisan make:seeder AreaSeeder

$areas = [
    'Asrama Nusantara', 'Asrama Putra', 'Masjid Utama', 'Gedung A', 'Gedung B',
    'Gedung SMP', 'Gedung IIS', 'Asrama Putri A', 'Asrama Putri B', 'Asrama Putri C'
];

foreach ($areas as $area) {
    Area::create(['name' => $area]);
}

```

14.2 Category Seeder

```

php artisan make:seeder CategorySeeder

$categories = [
    'Air & Sanitasi', 'Kelistrikan', 'AC & Pendingin', 'Furniture',
    'Bangunan', 'Kebersihan', 'Sound System', 'Lainnya'
];

foreach ($categories as $category) {
    Category::create(['name' => $category]);
}

```

14.3 DatabaseSeeder.php

```

public function run(): void
{
    $this->call([
        AreaSeeder::class,
        CategorySeeder::class,
    ]);
}

php artisan db:seed

```

15. Authentication - Login dan Logout

Karena layout utama sudah berbasis AdminLTE 3 + Bootstrap 4 + Blade, autentikasi dapat dibuat manual menggunakan fasilitas Auth Laravel.

```
php artisan make:controller Auth/LoginController

class LoginController extends Controller
{
    public function showLogin()
    {
        return view('auth.login');
    }

    public function login(Request $request)
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required']
        ]);

        if (Auth::attempt($credentials, $request->boolean('remember'))) {
            $request->session()->regenerate();
            return redirect()->intended('/dashboard');
        }

        return back()->withErrors([
            'email' => 'Email atau password tidak sesuai.'
        ]->onlyInput('email'));
    }

    public function logout(Request $request)
    {
        Auth::logout();
        $request->session()->invalidate();
        $request->session()->regenerateToken();
        return redirect('/login');
    }
}
```

15.1 Halaman Login

Buat **resources/views/auth/login.blade.php**. Gunakan layout login AdminLTE. Form harus menggunakan method POST dan @csrf, serta action route login.process.

```
<form method="POST" action="{{ route('login.process') }}">
    @csrf
    <input type="email" name="email" class="form-control" required>
    <input type="password" name="password" class="form-control" required>
    <button type="submit" class="btn btn-primary btn-block">Login</button>
</form>
```

15.2 Route Login

```
Route::get('/login', [LoginController::class, 'showLogin'])->name('login');
Route::post('/login', [LoginController::class, 'login'])->name('login.process');
Route::post('/logout', [LoginController::class, 'logout'])->name('logout');
```

15.3 Proteksi Route

```
Route::middleware('auth')->group(function () {
    // seluruh route aplikasi CityFix
});
```

16. Membuat User Admin Pertama

```
php artisan tinker

use App\Models\User;

User::create([
    'name' => 'CityFix Administrator',
    'email' => 'admin@cityfix.local',
    'password' => bcrypt('admin123'),
    'role' => 'admin'
```

```
]);
```

Gunakan password contoh hanya untuk development lokal. Jangan gunakan kredensial sederhana seperti ini di production.

17. Role Middleware

```
php artisan make:middleware RoleMiddleware

class RoleMiddleware
{
    public function handle(Request $request, Closure $next, ...$roles): Response
    {
        if (! auth()->check()) {
            return redirect()->route('login');
        }

        if (! in_array(auth()->user()->role, $roles)) {
            abort(403);
        }

        return $next($request);
    }
}
```

Register alias middleware di **bootstrap/app.php**:

```
$middleware->alias([
    'role' => RoleMiddleware::class,
]);
```

Contoh pembatasan:

```
Route::middleware(['auth', 'role:admin,verifier'])->group(function () {
    Route::resource('areas', AreaController::class);
});
```

18. Report Engine - Menyimpan Laporan

```
php artisan make:controller ReportController
```

18.1 Routes laporan

```
Route::get('/reports', [ReportController::class, 'index'])->name('reports.index');
Route::get('/reports/create', [ReportController::class, 'create'])->name('reports.create');
Route::post('/reports', [ReportController::class, 'store'])->name('reports.store');
Route::get('/reports/{report}', [ReportController::class, 'show'])->name('reports.show');
```

18.2 Method create()

```
public function create()
{
    $areas = Area::where('is_active', true)->orderBy('name')->get();
    $categories = Category::where('is_active', true)->orderBy('name')->get();
    return view('reports.create', compact('areas', 'categories'));
}
```

18.3 Form laporan

```
<form method="POST" action="{ route('reports.store') }" enctype="multipart/form-data">
    @csrf
    ...
</form>
```

Field minimal: photo, area_id, category_id, location_detail, urgency, dan description.

18.4 Method store()

```
public function store(Request $request)
{
    $validated = $request->validate([
        'area_id' => 'required|exists:areas,id',
    ]);
}
```

```

        'category_id' => 'required|exists:categories,id',
        'location_detail' => 'required|string|max:255',
        'description' => 'required|string',
        'urgency' => 'required|in:low,medium,high,emergency',
        'photo' => 'required|image|max:5120',
    ]);

    $photoPath = $request->file('photo')->store('reports','public');

    $report = Report::create([
        'report_number' => 'CF-'.now()->format('YmdHis'),
        'user_id' => auth()->id(),
        'area_id' => $validated['area_id'],
        'category_id' => $validated['category_id'],
        'location_detail' => $validated['location_detail'],
        'description' => $validated['description'],
        'urgency' => $validated['urgency'],
        'status' => 'reported',
        'photo' => $photoPath,
        'reported_at' => now(),
    ]);

    ReportHistory::create([
        'report_id' => $report->id,
        'user_id' => auth()->id(),
        'status' => 'reported',
        'note' => 'Laporan dibuat',
    ]);

    return redirect()->route('reports.show',$report)
        ->with('success','Laporan berhasil dibuat.');
```

19. File Storage

Foto kerusakan disimpan di storage/app/public/reports. Agar dapat diakses browser, jalankan:

```
php artisan storage:link
```

Menampilkan foto di Blade:

```

```

20. Daftar Laporan Berbasis Database

20.1 Method index()

```

public function index()
{
    $reports = Report::with(['area','category','reporter','technician'])
        ->latest()->paginate(15);

    return view('reports.index', compact('reports'));
}
```

20.2 Loop di Blade

```

@foreach($reports as $report)
<tr>
    <td>{{ $report->report_number }}</td>
    <td>{{ $report->created_at->format('d M Y H:i') }}</td>
    <td>{{ $report->area->name }}</td>
    <td>{{ $report->description }}</td>
    <td>{{ ucfirst($report->urgency) }}</td>
    <td>{{ ucfirst(str_replace('_', ' ', $report->status)) }}</td>
    <td><a href="{{ route('reports.show',$report) }}" class="btn btn-sm btn-info">Detail</a></td>
</tr>
@endforeach

{{ $reports->links() }}
```

21. Detail Laporan dan Progress Tracker

21.1 Method show()

```
public function show(Report $report)
{
    $report->load(['area', 'category', 'reporter', 'technician', 'histories.user']);
    return view('reports.show', compact('report'));
}
```

21.2 Progress timeline

```
@foreach($report->histories->sortBy('created_at') as $history)
    <strong>{{ ucfirst(str_replace('_', ' ', $history->status)) }}</strong><br>
    <small>{{ $history->created_at->format('d M Y H:i') }}</small>
    @if($history->note)<p>{{ $history->note }}</p>@endif
    <hr>
@endforeach
```

22. Assignment Teknisi

Ambil daftar teknisi:

```
$technicians = User::where('role', 'technician')->orderBy('name')->get();
```

Route assignment:

```
Route::post('/reports/{report}/assign', [ReportController::class, 'assign'])
    ->name('reports.assign');
```

Method assign():

```
public function assign(Request $request, Report $report)
{
    $validated = $request->validate([
        'assigned_to' => 'required|exists:users,id'
    ]);

    $report->update([
        'assigned_to' => $validated['assigned_to'],
        'status' => 'verified'
    ]);

    ReportHistory::create([
        'report_id' => $report->id,
        'user_id' => auth()->id(),
        'status' => 'verified',
        'note' => 'Laporan diverifikasi dan teknisi ditugaskan.'
    ]);

    return back()->with('success', 'Teknisi berhasil ditugaskan.');
```

23. Status Tracking

Workflow utama:

```
reported
|
verified
|
in_progress
|
completed
```

Status tambahan: waiting_material dan rejected.

```
Route::post('/reports/{report}/status', [ReportController::class, 'updateStatus'])
    ->name('reports.status');

public function updateStatus(Request $request, Report $report)
{

```

```

$validated = $request->validate([
    'status' => 'required|in:verified,in_progress,waiting_material,completed,rejected',
    'note' => 'nullable|string'
]);

$data = ['status' => $validated['status']];

if ($validated['status'] === 'in_progress') {
    $data['started_at'] = now();
}

if ($validated['status'] === 'completed') {
    $data['completed_at'] = now();
}

$report->update($data);

ReportHistory::create([
    'report_id' => $report->id,
    'user_id' => auth()->id(),
    'status' => $validated['status'],
    'note' => $validated['note'] ?? null
]);

return back()->with('success','Status berhasil diperbarui.');
```

24. Bukti Penyelesaian

Untuk status completed, tambahkan upload foto hasil perbaikan.

```

$request->validate([
    'completion_photo' => 'nullable|image|max:5120',
]);

$completionPhoto = null;
if ($request->hasFile('completion_photo')) {
    $completionPhoto = $request->file('completion_photo')
        ->store('completion','public');
}

ReportHistory::create([
    'report_id' => $report->id,
    'user_id' => auth()->id(),
    'status' => $validated['status'],
    'note' => $validated['note'] ?? null,
    'photo' => $completionPhoto
]);
```

Target tampilan timeline: Dilaporkan -> Diverifikasi -> Dalam Penanganan -> Selesai, dengan foto bukti pada langkah terakhir.

25. Dashboard Realtime

```

php artisan make:controller DashboardController

class DashboardController extends Controller
{
    public function index()
    {
        $totalReports = Report::count();
        $inProgress = Report::where('status','in_progress')->count();
        $completed = Report::where('status','completed')->count();
        $reported = Report::where('status','reported')->count();

        $latestReports = Report::with(['area','category'])
            ->latest()->limit(10)->get();

        return view('dashboard', compact(
            'totalReports','inProgress','completed','reported','latestReports'
        ));
    }
}
```

```
}
```

Update route dashboard agar menggunakan controller tersebut, lalu ganti angka dummy di Blade menjadi variabel seperti `{{ $totalReports }}`, `{{ $inProgress }}`, `{{ $completed }}`, dan `{{ $reported }}`.

26. Logout dan User Info

Di navbar AdminLTE tampilkan nama user aktif dan tombol logout.

```
{{ auth()->user()->name }}  
  
<form method="POST" action="{{ route('logout') }}">  
    @csrf  
    <button type="submit" class="btn btn-link nav-link">Logout</button>  
</form>
```

27. Role-based Menu

Sidebar sebaiknya berubah sesuai role pengguna.

Role	Menu/Fungsi Utama
Reporter	Dashboard, Buat Laporan, Laporan Saya
Verifier	Dashboard, Reports, Assignment, Monitoring
Technician	Dashboard, My Task, Update Status
Admin	Semua akses + Master Data + User

Contoh Blade:

```
@if(auth()->user()->role === 'admin')  
    <!-- menu master data -->  
@endif
```

28. Testing End-to-End

Buat minimal empat user: Reporter, Verifier, Technician, dan Admin.

Test	Skenario
01	Reporter login -> buat laporan -> upload foto -> submit
02	Verifier login -> buka laporan -> verifikasi -> assign technician
03	Technician login -> update Dalam Penanganan -> upload foto -> Selesai
04	Reporter login -> buka laporan -> lihat progress -> lihat bukti selesai

29. Struktur Project Setelah Tahap 2

```
app  
|- Http  
|   |- Controllers  
|   |   |- Auth/LoginController.php  
|   |   |- DashboardController.php  
|   |   |- ReportController.php  
|   |   |- AreaController.php  
|   |   `-- CategoryController.php  
|   `-- Middleware/RoleMiddleware.php  
|- Models  
|   |- User.php  
|   |- Area.php  
|   |- Category.php
```

```

|   |- Report.php
|   `-- ReportHistory.php
|
database
| - migrations
`- seeders
    |- AreaSeeder.php
    `-- CategorySeeder.php

resources/views
| - auth/login.blade.php
| - layouts/app.blade.php
| - reports/index.blade.php
| - reports/create.blade.php
| - reports/show.blade.php
`- dashboard.blade.php

routes/web.php

```

30. Checklist Tahap 2

No	Development	Status
1	Install/start MySQL	☐
2	Buat database cityfix	☐
3	Update .env	☐
4	Migration berjalan	☐
5	Tabel Area	☐
6	Tabel Category	☐
7	Tabel Reports	☐
8	Tabel Report History	☐
9	Seeder Area	☐
10	Seeder Category	☐
11	Login	☐
12	Logout	☐
13	User role	☐
14	Role middleware	☐
15	Form laporan tersimpan	☐
16	Upload foto	☐
17	Daftar laporan database	☐
18	Detail laporan	☐
19	Assignment PIC	☐
20	Status tracking	☐
21	Foto bukti selesai	☐
22	Dashboard realtime	☐
23	Role-based menu	☐

No	Development	Status
24	End-to-end testing	[]

31. Milestone Tahap 2

Milestone	Fokus	Output
2.1	Database Foundation	MySQL + Migration + Models + Seeder
2.2	User Access	Login + Logout + Role + Middleware
2.3	Core CityFix	Create Report + Upload Photo + List + Detail + History
2.4	Operational Workflow	Verification + Assign Technician + Update Status + Completion Photo + Rea

Urutan ini penting. Jangan membuat assignment teknisi sebelum user dan role tersedia, dan jangan membuat dashboard realtime sebelum report engine sudah menyimpan data dengan benar.

32. Titik Mulai yang Disarankan

Kerjakan dulu sampai **Milestone 2.1**: MySQL + migration + model + seeder. Setelah itu lanjutkan Milestone 2.2. Pendekatan bertahap akan jauh lebih mudah untuk troubleshooting daripada mengerjakan seluruh development sekaligus.

Setelah seluruh Tahap 2 selesai, CityFix sudah dapat diperlakukan sebagai **Minimum Viable Product (MVP)**: pelapor membuat laporan, sarpras memverifikasi, teknisi menangani, bukti perbaikan tersimpan, dan histori dapat dilihat kembali.